

P2500R1

C++ parallel algorithms and P2300

Published Proposal, 2023-05-18

This version:

<https://wg21.link/P2500R1>

Authors:

[Ruslan Arutyunyan](#) (Intel)

[Alexey Kukanov](#) (Intel)

Audience:

SG1, LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper provides the facilities to integrate [\[P2300R7\]](#) with C++ parallel algorithms

Table of Contents

- 1 Motivation**
- 2 Design overview**
 - 2.1 Design goals
 - 2.2 Combining a scheduler with a policy
 - 2.2.1 Why `scheduler`
 - 2.2.2 Alternative API
 - 2.3 Parallel algorithms are customizable functions
 - 2.4 Covering both "classic" and range algorithms
 - 2.4.1 Absence of serial range-based algorithms

2.5 Standard execution policies for range algorithms

3 Proposed API

4 API Overview

4.1 Possible implementations of a parallel algorithm

4.1.1 Customizable with `tag_invoke`

4.1.2 Customizable with language support

4.2 `execute_on`

4.3 `policy_aware_scheduler`

4.4 `execution_policy` concept

5 Further exploration

6 Revision History

6.1 R0 => R1

References

Normative References

Informative References

§ 1. Motivation

C++ parallel algorithms, together with executions policies, were a good start for supporting parallelism in the C++ standard. The C++ standard execution policies represent "how" a particular algorithm should be executed; in other words, they set semantical requirements to user callable objects passed to parallel algorithms. However, there is no explicit way to specify what hardware an algorithm should be executed on.

In the lack of better facilities in the C++ standard library, execution policies tend to be used to combine semantics of both "how" and "where" the code should be executed. Examples can be seen in [Thrust](#) and [oneDPL](#) libraries.

[P2300R7] introduces the `scheduler` concept that represents an execution context. Comparing to execution policies, it's a more flexible abstraction for answering "where" the code should be executed, because a `scheduler` could be tightly connected to the platform it sends work to.

As [P2300R7] progresses towards being added into C++26, we should answer the question how other parts of the C++ standard library would interoperate with schedulers/senders/receivers.

[\[P2214R2\]](#) outlined a plan to extend the standard Ranges library in C++23. The plan puts adding parallel overloads for the range algorithms into "Tier 2", among other motivating that by the need to carefully consider how these would work when [\[P2300R7\]](#) lands in the standard. To the best of our knowledge, nobody has yet approached this question.

This paper is targeted to C++26 and proposes the way for standard C++ algorithms to utilize [\[P2300R7\]](#) facilities.

§ 2. Design overview

§ 2.1. Design goals

The key question we need to address is how the API of C++ algorithms, including parallel and range based ones, should be extended or modified to express the notion that a certain algorithm should run in a certain execution context. The core semantics of an algorithm is expected to be preserved, except for any adjustments dictated by the need to execute work possibly in parallel by an unknown set of threads of execution. In particular, we think execution of the algorithms should remain synchronous, i.e. complete all the work upon return.

Another important design goal is to allow implementors of a particular execution context to also customize the implementation of C++ algorithms in that context. We consider this a must to provide best possible implementations for a given platform. At the same time, an algorithm should also have a default implementation, presumably expressed via other algorithms or some basic routines (see [§ 5 Further exploration](#)), allowing to customize only what is necessary.

§ 2.2. Combining a scheduler with a policy

To achieve the first goal, we propose to extend the approach of C++ parallel algorithms and allow in the place of an execution policy also passing a *policy-aware scheduler* that combines a policy and a representation of an execution context. This follows the existing practice of using a single argument to specify both "where" and "how" to execute an algorithm. It also forces binding a policy with a context prior to the algorithm invocation, allowing for better handling of possible mismatches between the two, e.g. in case the execution context cannot properly support the semantics of the policy, as well as for reuse of the resulting policy-aware scheduler instance.

An example declaration of `std::for_each` for the outlined approach would be:

```
template <policy_aware_scheduler Scheduler, typename ForwardIterator, typen  
void for_each(Scheduler&& sched, ForwardIterator first, ForwardIterator las
```

A `policy_aware_scheduler` is obtained with the `execute_on` function applied to a desired scheduler and a desired execution policy. Eventually, invoking a parallel algorithm to execute by a scheduler looks like:

```
std::for_each(std::execute_on(scheduler, std::execution::par), begin(data),
```

See [§ 4.3 policy_aware_scheduler](#) and [§ 4.2 execute_on](#) sections for more details.

§ 2.2.1. Why `scheduler`

The proposed API is blocking by design and behaves similarly to C++17 parallel algorithms. That means, when an algorithm returns the parallel execution is complete. The algorithm could build as complex a dependency graph within as it wants to, for which a `scheduler` allows to obtain as many senders as the algorithm needs for the implementation.

If we imagine that the algorithm takes a `sender`, it's unclear what to do then because that `sender` could represent any dependency chain built by the user, and all possible strategies of handling it we could imagine seem bad:

- We could ignore the `sender` and just obtain the `scheduler` from it, but that's likely not what users would expect
- We could run `sync_wait` on `sender` and then run the dependency graph that is built by the algorithm implementation, but in this case we lose the value `sync_wait` might return
- We could build the `sender` into the algorithm implementation chain, but again it's unclear what to do with the possible return value of the `sender`. For example it might return `int` while the algorithm semantically returns an iterator

Furthermore, from the customization perspective we are interested in execution context first that exactly represented by a `scheduler`.

§ 2.2.2. Alternative API

An alternative API might instead take both `scheduler` and `execution_policy` as function parameters.

```
template <scheduler Scheduler, execution_policy Policy, typename ForwardIter>
void for_each(Scheduler&& sched, Policy&& p, ForwardIterator first, ForwardIterator last)
```

However, in our opinion it complicates the signature for no good reason. The algorithm implementation would still first need to check if the scheduler can work with the execution policy, just on a later stage comparing to the preferred approach. Such a check would have to be redirected to the scheduler and/or the policy itself, and so would anyway require either something like § 4.2 `execute_on` or a member function defined by `schedulers` or by execution policies.

§ 2.3. Parallel algorithms are customizable functions

In line with the second design goal, we use the notion of *customizable functions* for parallel algorithms. It is essentially the same notion as proposed in [Language support for customisable functions § terminology](#), but without specific details. Similar to the algorithm function templates in `namespace std::ranges`, these cannot be found by argument-dependent lookup. In addition, these functions can be customized for a particular policy-aware scheduler. The implementation should invoke such a customization, if exists, otherwise execute a default generic implementation. That allows customizing every particular algorithm by `scheduler` vendors, if necessary.

We are not saying now which exactly customization mechanism will eventually be used, but it should be consistent across all parallel algorithms. The practical alternatives to consider are [std::execution § spec-func.tag_invoke](#) and [\[P2547R1\]](#). We would prefer to define the parallel algorithms in a way that does not depend on a particular customization mechanism, however that might be not practically possible due to the syntactical differences in how customizations are declared.

§ 2.4. Covering both "classic" and range algorithms

[\[P2500R0\]](#) suggested to only extend the "classic" C++17 parallel algorithms with a policy-aware scheduler, without touching the C++20 constrained algorithms over ranges. Besides being limited in scope, that also has several drawbacks:

- Keeping the existing algorithm names (`std::for_each` etc.) and yet allowing their customization requires us to:
 - Either redefine the names as customization point objects or as function objects supporting the `tag_invoke` mechanism. That would likely be considered as an ABI breaking change.
 - Or add function overloads constrained with `policy_aware_scheduler`, and require that they call new, specially defined customization point objects, like `std::for_each_cpo`. Making this for every algorithm would double the number of entities.
- The API with iterator pairs is more restrictive than with the iterator-and-sentinel pairs. One can pass two iterators as the arguments to range-based algorithms that take iterator and sentinel, while it's not possible to pass a sentinel instead of the second iterator to a "classic" algorithm.

In the current revision, we instead propose to define customizable algorithms with scheduling support in `namespace std::ranges`. Implementation-wise, that most likely means extending the existing function object types with new constrained overloads of `operator()`, which we think should not create any breaking changes. The algorithm functions in `namespace std` can then be supplemented with new overloads for `policy_aware_scheduler` that are required to call respective algorithms from `std::ranges`. This approach eliminates the drawbacks described above and also addresses the desire to support the execution semantics for the range-based algorithms. The consequence is that algorithms in `std` can be customized only via range-based algorithms. We think it's a reasonable tradeoff comparing to dozens of artificial customization points or potential ABI breaks.

§ 2.4.1. Absence of serial range-based algorithms

We understand that some range-based algorithms do not exist even as serial ones today. For example `<numeric>` does not have respective algorithms in `std::ranges`. It is supposed to be addressed either by this or by a complementary paper.

§ 2.5. Standard execution policies for range algorithms

Since this proposal addresses the problem of extending range algorithms to work with schedulers, we think it makes sense to address the lack of execution policy overloads for range algorithms as well. Such overloads can be safely added without any risk of conflict with the scheduler support, as an execution policy does not satisfy the requirements for a policy-aware scheduler, and vice versa.

At this point we do not, however, discuss how the appearance of schedulers may or should impact the execution rules for parallel algorithms specified in [\[algorithms.parallel.exec\]](#), and just assume that the same rules apply to the range algorithms with execution policies.

§ 3. Proposed API

Note that `std::ranges::for_each` and `std::for_each` are used as references. When the design is ratified, it will be applied to all parallel algorithms.

All the examples are also based on the `for_each` algorithms.

§ 4. API Overview

```
// Execution policy concept
template <typename ExecutionPolicy>
concept execution_policy = std::is_execution_policy_v<std::remove_cvref_t<ExecutionPolicy>>;

// Policy aware scheduler
template <typename S>
concept policy_aware_scheduler = scheduler<S> && requires (S s)
{
    typename S::base_scheduler_type;
    typename S::policy_type;
    { s.get_policy() } -> execution_policy;
};

// execute_on customization point
inline namespace /* unspecified */
{
    inline constexpr /* unspecified */ execute_on = /* unspecified */;
}

// std::ranges::for_each as an parallel algorithm example. Others can be done similarly.

// Policy-based API
template<execution_policy Policy, input_iterator I, sentinel_for<I> S, class Proj,
        indirectly_unary_invocable<projected<I, Proj>> Fun>
    constexpr ranges::for_each_result<I, Fun>
        ranges::for_each(Policy&& policy, I first, S last, Fun f, Proj proj = {});

template<execution_policy Policy, input_range R, class Proj = identity,
        indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun>
    constexpr ranges::for_each_result<borrowed_iterator_t<R>, Fun>
        ranges::for_each(Policy&& policy, R&& r, Fun f, Proj proj = {});

// Scheduler-based API
```

```

template<policy_aware_scheduler Scheduler, input_iterator I, sentinel_for<I> S>
    class Proj = identity, indirectly_unary_invocable<projected<I, Proj>> Fun>
    constexpr ranges::for_each_result<I, Fun>
        ranges::for_each(Scheduler sched, I first, S last, Fun f, Proj proj = Proj{}) {
template<policy_aware_scheduler Scheduler, input_range R, class Proj = identity,
    indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun>
    constexpr ranges::for_each_result<borrowed_iterator_t<R>, Fun>
        ranges::for_each(Scheduler sched, R&& r, Fun f, Proj proj = Proj{}) /*customization point*/

// "Classic" parallel algorithms with scheduler
template <policy_aware_scheduler Scheduler, typename ForwardIterator, typename Sentinel,
    void
        for_each(Scheduler&& sched, ForwardIterator first, ForwardIterator last, Sentinel last,

```

§ 4.1. Possible implementations of a parallel algorithm

Depending on a particular customization mechanism eventually chosen, a parallel algorithm can be implemented in one of the following ways.

In the current design all proposed APIs are customizable via one customization point, which is the overload that takes `I` and `S` (iterator and sentinel) because others can be redirected to that. We suppose that users want necessary algorithm being customized once and then having all its overloads automatically customized. It's unclear so far, why there should be such a flexibility that would allow customizing every particular overload individually but we are open for a discussion where people think it might be useful.

§ 4.1.1. Customizable with `tag_invoke`

```

// std::ranges::for_each possible implementation
namespace ranges
{
    namespace __detail
    {
        struct __for_each_fn
        {
            // ...
            // Existing serial overloads
            // ...

```

```

template<policy_aware_scheduler Scheduler, input_iterator I, sentinel_1
        class Proj = identity, indirectly_unary_invocable<projected<I,
constexpr for_each_result<I, Fun>
operator()(Scheduler sched, I first, S last, Fun f, Proj proj = {}) cor
{
    if constexpr (std::tag_invocable<__for_each_fn, Scheduler, It, S, C
    {
        std::tag_invoke(*this, sched, first, last, f, proj);
    }
    else
    {
        // default implementation
    }
}

template<policy_aware_scheduler Scheduler, input_range R, class Proj =
        indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fun
constexpr for_each_result<borrowed_iterator_t<R>, Fun>
operator()(Scheduler sched, R&& r, Fun f, Proj proj = {}) const
{
    return (*this)(sched, std::ranges::begin(r), std::ranges::end(r), f
}

}; // struct for_each
} // namespace __detail
inline namespace __for_each_fn_namespace
{
inline constexpr __detail::__for_each_fn for_each;
} // __for_each_fn_namespace
} // namespace ranges

```

§ 4.1.2. Customizable with language support

Here we assume that all `std::ranges::for_each` overloads, including ones that do not take a policy or a scheduler, are defined as `customizable` or `final` functions (in the sense of [P2547R1]).

We have not explored if it is practical to change the existing implementations of range algorithms in such a way.

```
// std::ranges::for_each possible implementation
namespace ranges
{
    // ...
    // Existing serial overloads
    // ...

    template<policy_aware_scheduler Scheduler, input_iterator I, sentinel_1
            class Proj = identity, indirectly_unary_invocable<projected<I,
    constexpr for_each_result<I, Fun>
    for_each(Scheduler sched, I first, S last, Fun f, Proj proj = {}) custo

    template<policy_aware_scheduler Scheduler, input_iterator I, sentinel_1
            class Proj = identity, indirectly_unary_invocable<projected<I,
    constexpr for_each_result<I, Fun>
    for_each(Scheduler&& sched, I first, S last, Fun f, Proj proj = {}) def
    {
        // default implementation
    }

    template<policy_aware_scheduler Scheduler, input_range R, class Proj =
            indirectly_unary_invocable<projected<iterator_t<R>, Proj>> Fur
    constexpr for_each_result<borrowed_iterator_t<R>, Fun>
    for_each(Scheduler sched, R&& r, Fun f, Proj proj = {})
    {
        return std::ranges::for_each(sched, std::ranges::begin(r), std::ran
    }
}
```

§ 4.2. `execute_on`

`execute_on` is the customization point that serves the purpose to tie `scheduler` and `execution_policy`.

It's up to a `scheduler` customization to check if it can work with the provided execution policy.

A possible implementation is:

```
namespace __detail
{
    struct __execute_on_fn {
        policy_aware_scheduler auto operator()(scheduler auto sched,
                                                execution_policy auto policy) const
        {
            return std::tag_invoke(*this, sched, policy);
        }
    }; // __execute_on_fn
} // namespace __detail

inline namespace __execute_on_fn_namespace
{
    inline constexpr __detail::__execute_on_fn execute_on;
} // __execute_on_fn_namespace
```

`execute_on` does not have a default implementation because it is generally impossible. Talking about execution policies we mean a broader set of those than just the standard ones (examples from Thrust and oneDPL libraries are referenced in [§ 1 Motivation](#)). It's hard to predict what should SYCL-based or CUDA-based scheduler do if the passed policy is something that the scheduler knows nothing about.

One could argue that for unsupported policies we should always fallback to sequential execution. But it's also incorrect in general case because the scheduler might represent an accelerator where sequential execution is not supported. Even on a CPU, falling back to sequential execution might be incorrect too, because the data might be allocated on an accelerator and be inaccessible for the CPU.

So it's up to the scheduler to decide whether to provide a fallback for unknown/unsupported policies or not, and if yes, to define what this fallback is doing.

§ 4.3. `policy_aware_scheduler`

`policy_aware_scheduler` is a concept that represents an entity that combines `scheduler` and `execution_policy`. It allows to get both execution policy type and execution policy object from the `policy_aware_scheduler` returned by `execute_on` call.

NOTE: `policy_type` and `execution_policy` object are not necessarily the same which `execute_on` was called with.

```
template <typename S>
concept policy_aware_scheduler = scheduler<S> && requires (S s) {
    typename S::base_scheduler_type;
    typename S::policy_type;
    { s.get_policy() } -> execution_policy;
};
```

See § 4.4 `execution_policy` concept for more details about `execution_policy` concept.

Customizations of the parallel algorithms can reuse the existing implementation (e.g., TBB-based, SYCL-based, CUDA-based) of parallel algorithms with `ExecutionPolicy` template parameter for "known" `base_scheduler_type` type.

§ 4.4. `execution_policy` concept

The execution policy concept is necessary if we want to constrain the return type of the `s.get_policy()` method for `policy_aware_scheduler`.

Since the scheduler tells "where" algorithms are executed and policies tell "how" algorithms are executed, we consider the set of policies currently defined in the `std::execution` namespace to be sufficient. So, the concept definition could look like:

```
template <typename ExecutionPolicy>
concept execution_policy = std::is_execution_policy_v<std::remove_cvref_t<ExecutionPolicy>>
```

We are open to make it more generic to allow adding custom policies for a particular scheduler, if somebody sees the value in it. For that case we either need to allow specializing `std::is_execution_policy` or to define another trait.

§ 5. Further exploration

The authors plan to explore how to specify a set of basic functions (a so-called "parallel backend") which parallel algorithms can be expressed with. It might be proposed in a separate paper based on the

analysis.

§ 6. Revision History

§ 6.1. R0 => R1

- Defined the API in terms of "customizable functions" instead of CPO
- Set range-based algorithms as the primary customization point for schedulers
- Proposed support for standard execution policies to range-based algorithms
- Defined scheduler-aware parallel algorithms in `namespace std` via constrained overloads redirecting to the range-based analogues
- Clarified behavior of `execute_on`

§ References

§ Normative References

[P2300R7]

Michał Dominiak; et al. *std::execution*. May 2023. URL: <https://wg21.link/P2300R7>

[P2547R1]

Lewis Baker, Corentin Jabot, Gašper Ažman. *Language support for customisable functions*. 16 July 2022. URL: <https://wg21.link/p2547r1>

§ Informative References

[P2214R2]

Barry Revzin, Conor Hoekstra, Tim Song. *A Plan for C++23 Ranges*. 18 February 2022. URL: <https://wg21.link/p2214r2>

[P2500R0]

Ruslan Arutyunyan. *C++17 parallel algorithms and P2300*. 15 October 2022. URL: <https://wg21.link/p2500r0>